实验 2 神经元模型、输入整合与阈值响应

2.1 实验目的

1. 理解 M-P 神经元的基本思想：掌握加权求和、阈值判断以及二值输出的基本过程，理解神经元“发放”与“不发放”的判定方式。

2. 观察人工神经元的输出特征：通过比较 sigmoid、ReLU、tanh 等激活函数的输出结果，理解现代人工神经元与 M-P 神经元的区别。

3. 理解突触输入整合作用：通过离散时间方式模拟 EPSP/IPSP 的叠加过程，观察兴奋输入、抑制输入以及阈值设置对输出的影响。

4. 观察简化脉冲神经元的发放现象：在给定的简化神经元模拟函数基础上，通过调整输入电流和阈值，分析神经元是否发放以及发放频率如何变化。

5. 掌握基础仿真与绘图方法：学会使用 numpy 和 matplotlib 编写简单仿真程序，并将实验结果绘制成图像进行分析。

2.2 实验说明

本次实验以“现象观察”和“参数调整”为主，不要求同学们从零搭建复杂的神经元模型。实验代码已经基本提供完成，同学们需要完成以下工作：

1. 修改给定代码中的少量参数；

2. 运行程序并观察输出结果；

3. 将结果绘制成图像；

4. 结合课堂内容分析实验现象。

在开始实验之前，请先激活实验 1 中创建的虚拟环境，并安装本次实验需要用到的库：

```
conda activate brain
pip install numpy matplotlib
```

如果校园网下载速度较慢，也可以使用镜像源，例如：

```
pip install numpy matplotlib -i https://pypi.tuna.tsinghua.edu.cn/simpl
e
```

一、M-P 神经元

M-P 神经元（McCulloch-Pitts neuron）是人工神经网络发展史中非常经典的神经元模型。它的核心思想非常简单：

- 神经元接收多个输入；
- 每个输入都对应一个权重；
- 对输入和权重做加权求和；
- 如果加权和大于或等于阈值，则输出 1；
- 否则输出 0。

它可以写成如下形式：

$y = 1$，当 $\sum(w_i x_i) >= \theta$

$y = 0$，当 $\sum(w_i x_i) < \theta$

下面的代码给出了两个样例输入，运行后请观察打印结果。

```python
import numpy as np



def mp_neuron(inputs, weights, threshold):
    weighted_sum = np.dot(inputs, weights)
    output = 1 if weighted_sum >= threshold else 0
    return weighted_sum, output



# 可以自行修改的参数
weights = np.array([0.7, 0.6, -0.5])
threshold = 1.0
```

```python
# 样例1：可以超过阈值
inputs_a = np.array([1.0, 1.0, 0.0])

# 样例2：不能超过阈值
inputs_b = np.array([1.0, 0.0, 1.0])

sum_a, output_a = mp_neuron(inputs_a, weights, threshold)
sum_b, output_b = mp_neuron(inputs_b, weights, threshold)

print("------ M-P 神经元实验 ------")
print("权重:", weights)
print("阈值:", threshold)
print()
print("样例 1 输入:", inputs_a)
print("样例 1 加权和:", round(sum_a, 3))
print("样例 1 输出:", output_a)
print()
print("样例 2 输入:", inputs_b)
print("样例 2 加权和:", round(sum_b, 3))
print("样例 2 输出:", output_b)
```

修改以下参数：

- `weights`
- `threshold`
- `inputs_a`
- `inputs_b`

完成以下现象观察，并在报告中分析结果：

1. 增大兴奋性输入对应的权重，观察输出是否更容易变为 1；
2. 增大抑制性输入对应的负权重，观察原本可能发放的输入是否变为不发放；
3. 提高阈值，观察同一组输入是否更难达到发放条件。

二、现代人工神经元

现代深度学习中的人工神经元通常也会先计算加权和：

$$z = \sum(w_i x_i) + b$$

然后再通过激活函数得到输出：

$$y = \text{激活函数}(z)$$

与 M-P 神经元不同，现代人工神经元的输出不一定只有 0 和 1，还可以是连续值。

下面我们比较三种常见激活函数：

- sigmoid
- ReLU
- tanh

运行下面代码，观察同一个输入在不同激活函数下的输出，并绘制各激活函数曲线。

```
import numpy as np
import matplotlib.pyplot as plt


def sigmoid(x):
    # 请根据公式自行实现:   sigmoid(x) = 1/(1+e^-x)
    raise NotImplementedError("请根据注释中的公式实现 sigmoid 函数")



def relu(x):
    return np.maximum(0.0, x)



def tanh(x):
    return np.tanh(x)



# 可以自行修改的参数
weights = np.array([0.8, -0.4, 0.6])
inputs = np.array([1.0, 0.5, 1.2])
bias = -0.2
```

```
z = np.dot(inputs, weights) + bias


print("------ 人工神经元实验 ------")
print("输入:", inputs)
print("权重:", weights)
print("偏置:", bias)
print("加权和 z =", round(z, 3))
print("sigmoid(z) =", round(float(sigmoid(np.array([z]))[0]), 3))
print("ReLU(z)    =", round(float(relu(np.array([z]))[0]), 3))
print("tanh(z)    =", round(float(tanh(np.array([z]))[0]), 3))


x = np.linspace(-5, 5, 400)


fig, axes = plt.subplots(1, 3, figsize=(12, 4))


axes[0].plot(x, sigmoid(x), color="tab:orange")
axes[0].set_title("Sigmoid")
axes[0].grid(alpha=0.3)


axes[1].plot(x, relu(x), color="tab:green")
axes[1].set_title("ReLU")
axes[1].grid(alpha=0.3)


axes[2].plot(x, tanh(x), color="tab:red")
axes[2].set_title("Tanh")
axes[2].grid(alpha=0.3)


for ax in axes:
    ax.set_xlabel("z")
    ax.set_ylabel("output")


fig.suptitle("不同激活函数的输出曲线", fontsize=14)
plt.tight_layout()
plt.show()
```

请先根据注释中的公式补全 sigmoid 函数，再运行整个脚本。

补全 sigmoid 函数后，再修改以下参数：

- `weights`
- `inputs`
- `bias`

在报告中分析以下问题：

1. `sigmoid` 和 `tanh` 都是连续变化的，它们的输出范围有什么区别？
2. 为什么 `ReLU` 在负半轴输出为 `0`？
3. 同样的输入经过不同激活函数后，输出结果为什么会不同？

三、EPSP/IPSP 与输入整合

课堂上已经介绍过：

- 兴奋性突触后电位 `EPSP` 会使膜电位上升；
- 抑制性突触后电位 `IPSP` 会使膜电位下降；
- 多个输入在时间上可以叠加；
- 叠加后的结果如果足够大，就更有可能达到阈值。

本实验不引入连续时间的膜电位微分方程，而是采用更容易理解的离散时间方法。也就是说，我们把时间划分为：

```
t = 0, 1, 2, 3, ...
```

我们假设：

- 一次兴奋输入会产生一个短时正响应波形，例如 `epsp = [0.8, 0.4, 0.2]`
- 一次抑制输入会产生一个短时负响应波形，例如 `ipsp = [-0.7, -0.4, -0.2]`

如果某一时刻发生了一次输入，那么它的影响会在接下来的几个离散时刻持续存在。多个输入的影响会叠加到同一个总响应曲线上。

下面的默认参数安排了三种典型现象：

1. 单次兴奋输入不能过阈值；

2. 两次相邻的兴奋输入叠加后可以过阈值；

3. 本来可能过阈值的一组兴奋输入，在加入抑制输入后被压低。

```
import numpy as np
import matplotlib.pyplot as plt



def add_wave(response, wave, start_time):
    for k, value in enumerate(wave):
        if start_time + k < len(response):
            response[start_time + k] += value



T = 20
time = np.arange(T)


# 可以自行修改的参数
epsp = np.array([0.8, 0.4, 0.2])
ipsp = np.array([-0.7, -0.4, -0.2])
threshold = 1.0


# 这些时刻发生兴奋输入和抑制输入
# t=2: 单次兴奋输入，不超过阈值
# t=6, 7: 相邻兴奋输入叠加，超过阈值
# t=12, 13: 如果没有抑制输入，本来有可能超过阈值
exc_times = [2, 6, 7, 12, 13]
inh_times = [13]


response = np.zeros(T)


for t0 in exc_times:
    add_wave(response, epsp, t0)


for t0 in inh_times:
    add_wave(response, ipsp, t0)
```

```
exc_idx = np.array(exc_times)
inh_idx = np.array(inh_times)
tick_positions = np.arange(0, T, 2)


output = (response >= threshold).astype(int)


fig, axes = plt.subplots(2, 1, figsize=(10, 7), sharex=True)


axes[0].stem(
    exc_idx,
    np.ones(len(exc_idx)),
    linefmt="tab:blue",
    markerfmt="bo",
    basefmt=" ",
)
axes[0].stem(
    inh_idx,
    -np.ones(len(inh_idx)),
    linefmt="tab:red",
    markerfmt="ro",
    basefmt=" ",
)
axes[0].set_title("输入事件时刻（蓝色为兴奋输入，红色为抑制输入）")
axes[0].set_ylabel("event")
axes[0].grid(alpha=0.3)


axes[1].plot(time, response, marker="o", label="总响应 response", color
="tab:purple")
axes[1].axhline(threshold, color="tab:green", linestyle="--", label="th
reshold")
axes[1].step(time, output, where="mid", label="二值输出 output", color="
tab:orange")
axes[1].set_title("EPSP/IPSP 叠加后的总响应")
axes[1].set_xlabel("time step")
axes[1].set_ylabel("response")
axes[1].set_xticks(tick_positions)
axes[1].set_xlim(-0.5, T - 0.5)
axes[1].grid(alpha=0.3)
axes[1].legend()


plt.tight_layout()
plt.show()
```

修改以下参数：

- `epsp`
- `ipsp`
- `exc_times`
- `inh_times`
- `threshold`

完成以下操作，并在报告中分析结果：

1. 删除 `inh_times = [13]` 中的抑制输入，观察后半段的总响应是否更容易过阈值；
2. 增大 `epsp` 的幅值，观察兴奋输入是否更容易触发输出；
3. 增大 `ipsp` 的绝对值，观察抑制作用是否更明显；
4. 调高或调低阈值，观察输出结果的变化。

四、简化的脉冲神经元

为了帮助大家更直观地理解“输入越强越容易发放”“阈值越低越容易发放”的现象，下面给出一个简化的脉冲神经元模拟函数。

这里需要特别说明：

- 这个函数只是一个用于观察现象的简化模型；
- 本实验不要求大家掌握它的推导过程；
- 重点是通过调节参数，观察神经元的膜电位变化和发放情况。

在下面的代码中：

- `input_current` 表示持续输入电流；
- `threshold` 表示发放阈值；
- `decay` 表示膜电位在每个时间步上的保留比例；

- 当膜电位达到阈值后，程序会记一次脉冲发放，并在下一步计算时从 reset_value 重新开始积累。

```
import numpy as np
import matplotlib.pyplot as plt


def simulate_neuron(input_current, threshold, decay=0.85, T=50, reset_v
alue=0.0):
    voltage = np.zeros(T)
    spikes = np.zeros(T)


    for t in range(1, T):
        # 1. 决定当前步的起点：如果上一步发射了脉冲，当前计算就从 reset_valu
e 开始
        if spikes[t - 1] == 1:
            prev_v = reset_value
        else:
            prev_v = voltage[t - 1]


        # 2. 积分与衰减
        voltage[t] = decay * prev_v + input_current


        # 3. 触发判断（只记录脉冲，不立刻强行抹除当前电压）
        if voltage[t] >= threshold:
            spikes[t] = 1


    return voltage, spikes


def main():
    T = 50
    time = np.arange(T)


    # 情况 1：相同阈值下，不同输入电流
    threshold_same = 1.0
    current_small = 0.12
    current_large = 0.28


    voltage_small, spikes_small = simulate_neuron(current_small, thresh
old_same, decay=0.85, T=T)
    voltage_large, spikes_large = simulate_neuron(current_large, thresh
```

```
old_same, decay=0.85, T=T)


    # 情况 2：相同输入电流下，不同阈值
    input_current_same = 0.22
    threshold_low = 0.75
    threshold_high = 1.30


    voltage_low, spikes_low = simulate_neuron(input_current_same, thres
hold_low, decay=0.85, T=T)
    voltage_high, spikes_high = simulate_neuron(input_current_same, thr
eshold_high, decay=0.85, T=T)


    fig, axes = plt.subplots(2, 2, figsize=(12, 8), sharex=True)


    axes[0, 0].plot(time, voltage_small, label=f"I={current_small}", co
lor="tab:blue")
    axes[0, 0].plot(time, voltage_large, label=f"I={current_large}", co
lor="tab:orange")
    axes[0, 0].axhline(threshold_same, linestyle="--", color="tab:green
", label="threshold")
    axes[0, 0].set_title("相同阈值下的膜电位变化")
    axes[0, 0].set_ylabel("voltage")
    axes[0, 0].grid(alpha=0.3)
    axes[0, 0].legend()


    axes[1, 0].step(time, spikes_small, where="mid", label=f"I={current
_small}", color="tab:blue")
    axes[1, 0].step(time, spikes_large, where="mid", label=f"I={current
_large}", color="tab:orange")
    axes[1, 0].set_title("相同阈值下的脉冲输出")
    axes[1, 0].set_xlabel("time step")
    axes[1, 0].set_ylabel("spike")
    axes[1, 0].grid(alpha=0.3)
    axes[1, 0].legend()


    axes[0, 1].plot(time, voltage_low, label=f"threshold={threshold_low}
", color="tab:red")
    axes[0, 1].plot(time, voltage_high, label=f"threshold={threshold_hi
gh}", color="tab:purple")
    axes[0, 1].axhline(threshold_low, linestyle="--", color="tab:red",
alpha=0.6)
    axes[0, 1].axhline(threshold_high, linestyle="--", color="tab:purpl
e", alpha=0.6)
```

```
    axes[0, 1].set_title("相同输入电流下的膜电位变化")
    axes[0, 1].set_ylabel("voltage")
    axes[0, 1].grid(alpha=0.3)
    axes[0, 1].legend()


    axes[1, 1].step(time, spikes_low, where="mid", label=f"threshold={t
hreshold_low}", color="tab:red")
    axes[1, 1].step(time, spikes_high, where="mid", label=f"threshold=
{threshold_high}", color="tab:purple")
    axes[1, 1].set_title("相同输入电流下的脉冲输出")
    axes[1, 1].set_xlabel("time step")
    axes[1, 1].set_ylabel("spike")
    axes[1, 1].grid(alpha=0.3)
    axes[1, 1].legend()


    plt.tight_layout()
    plt.show()



if __name__ == "__main__":
    main()
```

修改以下参数：

- current_small
- current_large
- threshold_same
- input_current_same
- threshold_low
- threshold_high
- decay

在报告中分析以下问题：

1. 在阈值相同的情况下，为什么较大的输入电流更容易使神经元发放；
2. 在输入电流相同的情况下，为什么较低的阈值更容易产生脉冲；

3. `decay` 变大或变小时，膜电位积累速度和发放频率会发生什么变化。

选做内容：

- 将输入电流改成随时间变化的数组，而不是固定常数；
- 使用 `SpikingJelly` 等类脑计算库复现类似现象。

2.3 实验要求

1. 在 `brain` 虚拟环境中安装并导入 `numpy` 和 `matplotlib`。
2. 运行本实验讲义中的四部分代码，保证程序能够正常输出结果。
3. 至少提交 3 张图像，包含以下内容：
    - 不同激活函数的输出比较图；
    - `EPSP/IPSP` 叠加与阈值响应图；
    - 简化的脉冲神经元中不同电流或不同阈值条件下的膜电位图、脉冲图。
4. 提交一份实验代码文件包。
5. 提交一份简短实验报告或结果说明，报告中必须包含图像和对应分析文字，并覆盖前文要求完成的现象观察、操作和问题分析。**选做内容及其他个人理解和分析，根据质量酌情打分。**

实验报告中回答以下问题：

1. 在本实验中，哪些参数会让神经元更容易达到阈值？为什么？
2. 抑制输入加入后，总响应曲线发生了什么变化？
3. M-P 神经元、现代人工神经元和脉冲式输出在表现形式上有什么不同？
4. 当输入电流增大或阈值减小时，神经元的发放现象如何变化？

提交内容如下：

- 实验代码文件，依次命名为 `exp2_1.py/exp2_2.py` 等；
- 一份实验报告；
- 报告中应包含关键图像、参数设置以及对实验现象的分析。

本次实验的重点是通过参数调整、图像观察和结果分析，加深对神经元模型、突触输入整合以及阈值响应机制的理解。